

INSTITUTO FEDERAL DA PARAÍBA
CAMPUS CAMPINA GRANDE
CURSO DE ENGENHARIA DA COMPUTAÇÃO

RYAN SOARES NUNES JORVINO

ESTUDO DIRIGIDO – ETAPA 4
AUTOMAÇÃO INDUSTRIAL E TECNOLOGIAS

Disciplina: Controle e Automação I
Professor: Dr. Moacy Pereira da Silva

Campina Grande – PB
Junho de 2026

1 INTRODUÇÃO

As três primeiras etapas deste estudo dirigido percorreram um caminho coerente: dos fundamentos teóricos de sistemas dinâmicos (Etapa 1), passando pela modelagem e simulação de sistemas físicos (Etapa 2), até o projeto de controle em malha fechada com o controlador PID (Etapa 3). Em todas elas, o foco esteve na análise matemática e na simulação computacional do comportamento de um sistema individual — particularmente o motor de corrente contínua.

Esta Etapa 4 amplia o horizonte para a escala industrial. Na prática, sistemas controlados não operam isoladamente: eles fazem parte de plantas industriais complexas, com dezenas ou centenas de sensores, atuadores, motores e processos interligados, todos coordenados por sistemas de automação. O controle PID de um motor, estudado na Etapa 3, é apenas um componente dentro de uma arquitetura muito maior, que envolve controladores lógicos programáveis (CLPs), sistemas supervisórios, interfaces homem-máquina e redes de comunicação industrial.

O objetivo desta etapa é conectar os conceitos de controle estudados com as tecnologias reais empregadas na indústria. O documento inicia com os fundamentos da automação industrial e a descrição de seus elementos essenciais, segue com a investigação das linguagens de programação de CLPs definidas pela norma IEC 61131-3, apresenta uma aplicação prática desenvolvida no ambiente OpenPLC o comando automatizado do motor CC estudado nas etapas anteriores e conclui com a discussão de tecnologias avançadas como o controle preditivo e a aplicação de inteligência artificial em controle e automação.

2 FUNDAMENTOS DE AUTOMAÇÃO INDUSTRIAL

A automação industrial é o emprego de sistemas de controle — como CLPs, computadores e robôs — e de tecnologias da informação para operar processos e máquinas com mínima intervenção humana. Seu objetivo é aumentar a produtividade, a qualidade e a segurança, ao mesmo tempo em que reduz custos e a exposição de operadores a ambientes perigosos ou tarefas repetitivas.

A evolução da automação é frequentemente descrita em termos de revoluções industriais. A primeira introduziu a mecanização a vapor; a segunda, a produção em massa com energia elétrica; a terceira, a automação eletrônica e os CLPs, a partir da década de 1960.

Atualmente vive-se a chamada Indústria 4.0, caracterizada pela integração de sistemas ciberfísicos, Internet das Coisas Industrial (IIoT), computação em nuvem, big data e inteligência artificial, criando fábricas inteligentes capazes de auto-otimização e manutenção preditiva.

Do ponto de vista estrutural, um sistema de automação industrial é tradicionalmente organizado em uma hierarquia de níveis, conhecida como pirâmide da automação. Na base estão os sensores e atuadores (nível de campo); acima deles, os CLPs e controladores (nível de controle); em seguida, os sistemas supervisórios SCADA (nível de supervisão); depois, os sistemas de execução da manufatura MES (nível de gerenciamento); e no topo, os sistemas de gestão empresarial ERP (nível corporativo). Cada nível comunica-se com os adjacentes, formando um fluxo de informação que vai do chão de fábrica à gestão estratégica.

3 ELEMENTOS DA AUTOMAÇÃO INDUSTRIAL

3.1 Sensores Industriais

Os sensores são os elementos que percebem o estado do processo, convertendo grandezas físicas em sinais elétricos que o controlador pode interpretar. São os olhos e ouvidos do sistema de automação. Entre os tipos mais comuns estão os sensores de proximidade (indutivos, capacitivos e ópticos), que detectam a presença de objetos sem contato físico; sensores de temperatura (termopares, termorresistências PT100); sensores de pressão; sensores de nível; e encoders, que medem posição e velocidade angular — exatamente o tipo de sensor que seria acoplado ao motor CC para realimentar o controle de velocidade estudado na Etapa 3.

3.2 Controlador Lógico Programável (CLP)

O Controlador Lógico Programável (CLP, ou PLC em inglês) é o cérebro da automação no nível de chão de fábrica. Trata-se de um computador industrial robusto, projetado para operar em ambientes hostis (poeira, vibração, temperatura, ruído elétrico) e para executar, de forma cíclica e determinística, um programa de controle que lê entradas, processa a lógica e atualiza saídas.

O funcionamento do CLP baseia-se no ciclo de varredura (scan cycle), composto por três fases repetidas continuamente: leitura das entradas (estado dos sensores), execução do

programa (processamento da lógica) e atualização das saídas (acionamento dos atuadores). Esse ciclo se repete em intervalos da ordem de milissegundos, garantindo resposta rápida e previsível — característica essencial para aplicações de controle em tempo real.

3.3 Sistemas Supervisórios (SCADA)

Os sistemas SCADA (Supervisory Control and Data Acquisition) operam no nível de supervisão, acima dos CLPs. Sua função é coletar dados de múltiplos controladores distribuídos pela planta, apresentá-los de forma centralizada a operadores, armazenar históricos para análise, gerar alarmes e permitir o comando remoto de processos. Um sistema SCADA típico inclui telas gráficas que representam a planta em tempo real, gráficos de tendências, registros de eventos e relatórios gerenciais. É o que permite que um único operador, de uma sala de controle, supervisione uma fábrica inteira.

3.4 Interface Homem-Máquina (IHM)

A Interface Homem-Máquina (IHM, ou HMI) é o ponto de contato direto entre o operador e a máquina ou processo. Tipicamente é uma tela sensível ao toque instalada junto ao equipamento, que exibe informações de estado (velocidades, temperaturas, contadores, alarmes) e permite comandos locais (partida, parada, ajuste de parâmetros). Enquanto o SCADA tem caráter centralizado e supervisório, a IHM é local e operacional. No exemplo do motor CC, uma IHM poderia exibir a velocidade atual, o número de partidas e o tempo de operação — precisamente as variáveis monitoradas pelo programa de supervisão apresentado adiante.

3.5 Protocolos de Comunicação Industrial

Para que sensores, CLPs, IHMs e supervisórios troquem informações de forma padronizada e confiável, utilizam-se protocolos de comunicação industrial. Entre os mais difundidos estão:

- **Modbus:** um dos protocolos mais antigos e simples, amplamente suportado, nas variantes Modbus RTU (serial) e Modbus TCP (sobre Ethernet). É inclusive o protocolo nativo do OpenPLC usado na aplicação prática desta etapa.

- **PROFIBUS e PROFINET:** padrões amplamente adotados na indústria, o primeiro serial e o segundo sobre Ethernet industrial, oferecendo alta velocidade e determinismo.
- **EtherNet/IP:** protocolo industrial sobre Ethernet padrão, comum em equipamentos de fabricantes norte-americanos.
- **OPC UA:** padrão moderno e independente de fabricante, voltado à interoperabilidade e à integração com a Indústria 4.0, permitindo comunicação segura desde o chão de fábrica até a nuvem.

4 LINGUAGENS DE PROGRAMAÇÃO DE CLPs

A programação de CLPs é padronizada pela norma internacional IEC 61131-3, que define cinco linguagens: três gráficas (Ladder Diagram, Function Block Diagram e Sequential Function Chart) e duas textuais (Structured Text e Instruction List). Esta seção investiga as duas mais relevantes e utilizadas: o Ladder Diagram e o Structured Text, ambas empregadas na aplicação prática desta etapa.

4.1 Ladder Diagram (LD)

O Ladder Diagram, ou diagrama de contatos, é a linguagem mais tradicional e difundida na automação industrial. Sua notação gráfica deriva diretamente dos diagramas elétricos de comando a relés, o que a tornou intuitiva para os técnicos e eletricitistas que faziam a transição da lógica eletromecânica para os CLPs nas décadas de 1970 e 1980.

Um programa Ladder é organizado em linhas horizontais chamadas degraus (rungs), delimitadas por duas barras verticais que representam a alimentação elétrica. Cada rung contém contatos (que representam condições de entrada) à esquerda e bobinas (que representam saídas) à direita. Os contatos podem ser normalmente abertos (que conduzem quando a variável é verdadeira) ou normalmente fechados (que conduzem quando a variável é falsa). A lógica é avaliada como o fluxo de energia da barra esquerda à direita: se houver um caminho contínuo de contatos fechados até a bobina, a saída é energizada.

As principais vantagens do Ladder são a clareza visual para lógica booleana (intertravamentos, selos, sequenciamentos), a facilidade de depuração — pois é possível ver o fluxo de energia em tempo real — e a familiaridade da equipe técnica. Sua limitação aparece

em tarefas que envolvem cálculos matemáticos complexos, manipulação de dados ou algoritmos elaborados, nos quais a representação gráfica torna-se verbosa e pouco prática.

4.2 Structured Text (ST)

O Structured Text é uma linguagem textual de alto nível, com sintaxe semelhante à das linguagens de programação tradicionais como Pascal e C. Suporta estruturas de controle de fluxo (IF/THEN/ELSE, FOR, WHILE, CASE), operações aritméticas e lógicas, atribuições e chamadas de funções e blocos funcionais.

Por sua natureza textual e estruturada, o ST é a linguagem mais adequada para implementar algoritmos complexos, cálculos matemáticos, processamento de dados, laços e lógica condicional sofisticada — tarefas que seriam extremamente trabalhosas em Ladder. Um controlador PID, por exemplo, é tipicamente implementado em ST, dada a necessidade de operações aritméticas com os termos proporcional, integral e derivativo. Em contrapartida, para lógica de intertravamento puramente booleana, o Ladder costuma ser mais legível e direto.

Na prática industrial, as duas linguagens são frequentemente combinadas em um mesmo projeto: o Ladder para a lógica de comando e segurança, e o Structured Text para os cálculos e a lógica de supervisão. Essa combinação é justamente a abordagem adotada na aplicação prática desenvolvida a seguir.

5 APLICAÇÃO PRÁTICA: COMANDO DO MOTOR CC NO OPENPLC

Para demonstrar concretamente os conceitos apresentados, foi desenvolvida uma aplicação no OpenPLC Editor — uma ferramenta gratuita e de código aberto que implementa um ambiente de programação de CLP conforme a norma IEC 61131-3, com suporte a simulação. A aplicação consiste no sistema de comando e supervisão do motor de corrente contínua estudado nas etapas anteriores, agora visto sob a ótica da automação industrial: o motor que foi modelado e controlado por PID passa a ser comandado por um CLP, com lógica de partida, proteção e sinalização.

5.1 Descrição do Sistema

O sistema implementa o comando clássico de partida e parada de um motor industrial, com os seguintes elementos de entrada e saída:

- **BotaoLiga (entrada %IX0.0):** botoeira de partida, normalmente aberta.
- **BotaoDesliga (entrada %IX0.1):** botoeira de parada.
- **ReleTermico (entrada %IX0.2):** contato do relé de proteção térmica, que desliga o motor em caso de sobrecarga.
- **Motor (saída %QX0.0):** aciona o contator que liga o motor.
- **LampVerde (saída %QX0.1):** sinalização de motor em operação.
- **LampVermelha (saída %QX0.2):** sinalização de motor parado.

5.2 Programa em Ladder Diagram

A lógica de comando foi implementada em Ladder, organizada em três degraus, conforme mostra a Figura 1.

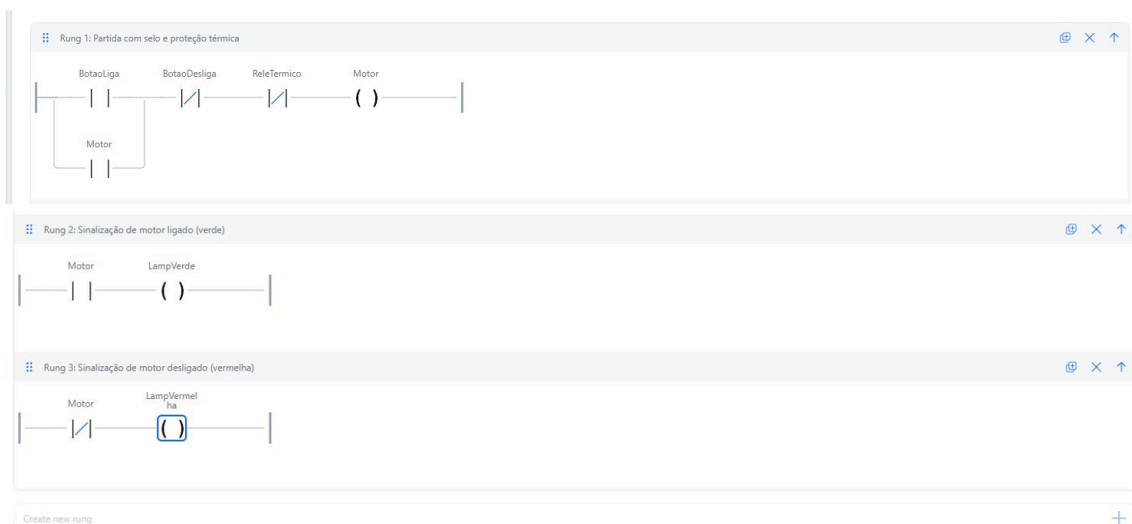


Figura 1 – Programa em Ladder Diagram para comando do motor CC, com os três degraus: partida com selo e proteção, sinalização verde e sinalização vermelha.

O primeiro degrau implementa o coração da lógica: a partida com selo e proteção. O motor é acionado quando o BotaoLiga é pressionado, desde que o BotaoDesliga não esteja acionado e o ReleTermico não tenha disparado (ambos representados por contatos normalmente fechados, que conduzem em condição normal). O elemento essencial deste degrau é o contato Motor em paralelo com o BotaoLiga — o chamado selo ou contato de retenção. Uma vez energizada a bobina Motor, esse contato se fecha e mantém o caminho de energia mesmo após o operador soltar o botão de partida. É o mecanismo que permite que um botão momentâneo produza um acionamento permanente.

O segundo degrau aciona a lâmpada verde sempre que o motor está ligado, por meio de um contato normalmente aberto da variável Motor. O terceiro degrau aciona a lâmpada vermelha quando o motor está desligado, utilizando um contato normalmente fechado da variável Motor — de modo que a sinalização vermelha permanece acesa em repouso e se apaga assim que o motor entra em operação. Os dois degraus de sinalização demonstram, de forma simples, o uso complementar de contatos normais e negados.

5.3 Programa em Structured Text

Para complementar a lógica de comando com uma camada de supervisão, foi desenvolvido um segundo programa em Structured Text. Aproveitando a aptidão do ST para cálculos e manipulação de dados, este programa conta o número de partidas do motor e acumula o tempo total de operação — informações típicas que seriam exibidas em uma IHM ou registradas por um sistema SCADA para fins de manutenção. A Figura 2 mostra o código e a declaração de variáveis.

Description :

Class Filter: All

+

-

↑

↓

🔍

#	Name	Class	Type	Location	Initial Value	Documentation	Debug
0	Motor	Input	BOOL				🔍
1	MotorAnterior	Local	BOOL				🔍
2	ContadorPartidas	Output	INT				🔍
3	TempoOperacao	Output	TIME				🔍
4	TimerOp	Local	TON				🔍

```

1  (* Supervisão do motor CC: conta partidas e mede tempo de operação *)
2
3  (* Detecta borda de subida: nova partida do motor *)
4  IF Motor AND NOT MotorAnterior THEN
5      ContadorPartidas := ContadorPartidas + 1;
6  END_IF;
7
8  MotorAnterior := Motor;
9
10 (* Temporizador acumula tempo enquanto o motor está ligado *)
11 TimerOp(IN := Motor, PT := T#1000h);
12 TempoOperacao := TimerOp.ET;

```

Figura 2 – Programa em Structured Text para supervisão do motor: contagem de partidas e medição do tempo de operação.

O código emprega dois recursos importantes do Structured Text. O primeiro é a detecção de borda de subida: a condição IF Motor AND NOT MotorAnterior identifica o exato instante em que o motor passa de desligado para ligado, incrementando o contador de partidas apenas uma vez por acionamento. A variável MotorAnterior, atualizada ao final de cada ciclo, armazena o estado anterior para a comparação. O segundo recurso é o uso de um bloco funcional temporizador (TON), que acumula o tempo durante o qual o motor permanece ligado, disponibilizando-o na variável TempoOperacao. Esse tipo de lógica,

simples em ST, seria consideravelmente mais verbosa e menos clara se implementada em Ladder — ilustrando bem a complementaridade entre as duas linguagens discutida na Seção 4.

5.4 Simulação e Validação

O projeto foi compilado e executado no simulador integrado do OpenPLC, que permite forçar os valores das entradas e observar, em tempo real, o estado dos contatos e bobinas — destacados em verde quando energizados. As Figuras 3 e 4 documentam os dois estados operacionais do sistema.

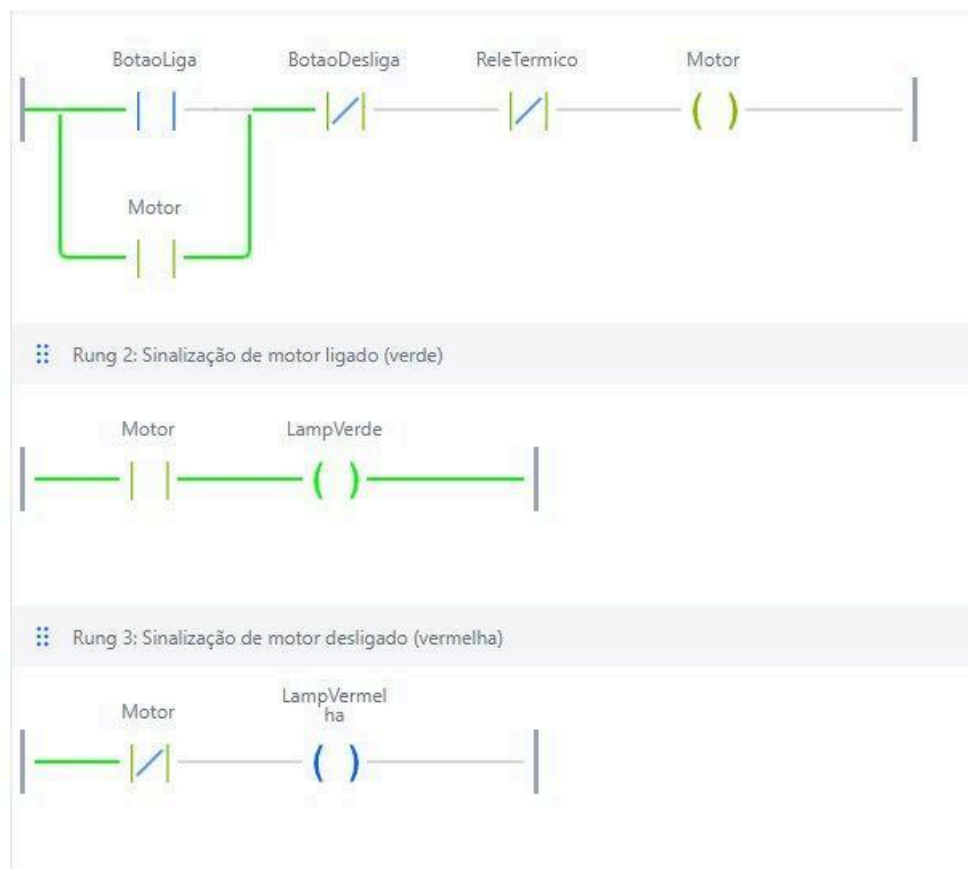


Figura 3 – Simulação com o motor ligado: o caminho de energia do primeiro degrau está energizado (verde) até a bobina Motor, com o selo ativo, e a lâmpada verde acesa.

Na Figura 3, observa-se o sistema após o acionamento do BotaoLiga. O caminho do primeiro degrau está completamente energizado (em verde) até a bobina Motor, e o contato de selo, também em verde, comprova que a retenção está ativa — o motor permaneceria ligado mesmo com o botão solto. O segundo degrau está energizado, acendendo a lâmpada verde de sinalização de operação.

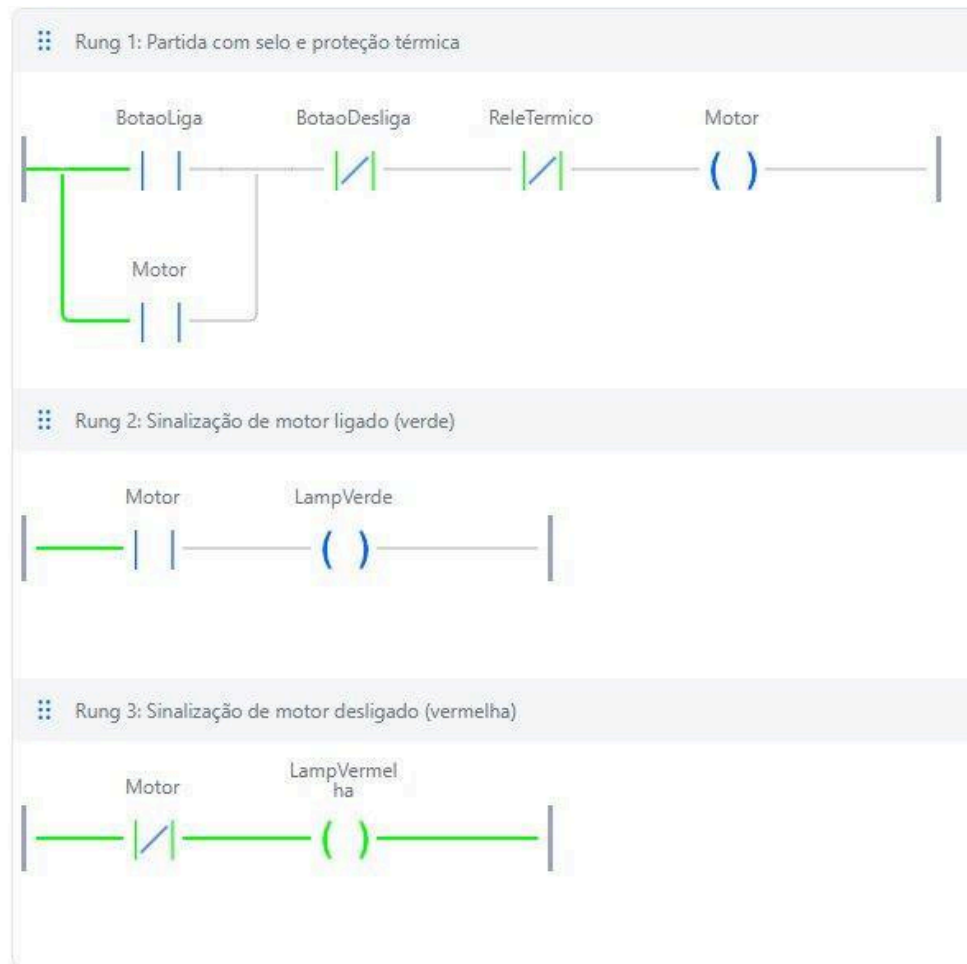


Figura 4 – Simulação com o motor desligado: o terceiro degrau está energizado até a lâmpada vermelha, indicando o estado de repouso.

Na Figura 4, com o motor desligado, o primeiro degrau não conduz até a bobina Motor (que permanece desenergizada) e o terceiro degrau está energizado em verde até a lâmpada vermelha, sinalizando corretamente o estado de repouso do sistema. A comutação entre os dois estados, comandada pelas botoeiras e pela proteção térmica, valida o funcionamento correto da lógica de comando implementada.

6 TEMAS AVANÇADOS EM CONTROLE E AUTOMAÇÃO

6.1 Controle Preditivo (MPC)

O Controle Preditivo Baseado em Modelo (MPC, Model Predictive Control) representa uma evolução significativa em relação ao controle PID. Em vez de reagir apenas ao erro atual, o MPC utiliza um modelo matemático da planta para prever seu comportamento futuro ao longo de um horizonte de tempo, e calcula a sequência de ações de controle que

otimiza um critério de desempenho, respeitando restrições explícitas — como limites de atuadores, faixas de segurança e especificações de qualidade.

A grande vantagem do MPC é justamente a capacidade de lidar nativamente com restrições e com sistemas de múltiplas entradas e saídas (MIMO), nos quais várias variáveis interagem. Por isso, é amplamente empregado em processos complexos das indústrias petroquímica, de refino e de papel e celulose. Sua principal limitação é o custo computacional: a cada ciclo, é necessário resolver um problema de otimização, o que exige hardware mais poderoso que o de um PID convencional. O modelo do motor CC desenvolvido neste estudo dirigido, com sua função de transferência conhecida, seria precisamente o ponto de partida para projetar um controlador MPC dessa planta.

6.2 Inteligência Artificial em Controle e Automação

A aplicação de inteligência artificial e aprendizado de máquina à automação industrial é uma das fronteiras mais ativas da área, central à Indústria 4.0. Diversas vertentes merecem destaque:

- **Manutenção preditiva:** algoritmos de aprendizado de máquina analisam dados de sensores (vibração, temperatura, corrente) para prever falhas de equipamentos antes que ocorram, reduzindo paradas não planejadas. No caso de um motor, padrões na corrente da armadura podem antecipar desgaste de rolamentos ou problemas de isolamento.
- **Controle inteligente:** controladores baseados em lógica fuzzy, redes neurais ou aprendizado por reforço podem controlar sistemas não-lineares ou de difícil modelagem, nos quais o PID clássico tem desempenho limitado.
- **Visão computacional:** sistemas de inspeção visual automática realizam controle de qualidade, detecção de defeitos e classificação de produtos em linhas de produção, com velocidade e consistência superiores à inspeção humana.
- **Otimização de processos:** algoritmos analisam grandes volumes de dados de produção para identificar ajustes de parâmetros que maximizam eficiência energética, throughput ou qualidade.

É importante observar que essas tecnologias avançadas não substituem os fundamentos estudados nas etapas anteriores — pelo contrário, apoiam-se neles. Um sistema de manutenção preditiva precisa compreender a dinâmica do motor; um controlador

inteligente ainda lida com polos, estabilidade e resposta temporal. A base teórica construída ao longo deste estudo dirigido é o alicerce sobre o qual essas técnicas modernas se erguem.

7 CONCLUSÃO

Esta quarta etapa do estudo dirigido ampliou o escopo da disciplina, conectando os conceitos de controle estudados nas etapas anteriores com as tecnologias reais da automação industrial. Partiu-se dos fundamentos e da pirâmide da automação, passou-se pela descrição de seus elementos essenciais sensores, CLPs, sistemas SCADA, IHMs e protocolos de comunicação e investigaram-se as duas principais linguagens de programação de CLPs definidas pela norma IEC 61131-3: o Ladder Diagram e o Structured Text.

A aplicação prática desenvolvida no OpenPLC deu concretude a esses conceitos. O comando do motor CC o mesmo sistema modelado na Etapa 2 e controlado por PID na Etapa 3 foi implementado sob a ótica da automação industrial, com lógica de partida com selo, proteção térmica e sinalização em Ladder, complementada por uma rotina de supervisão em Structured Text. A simulação validou o funcionamento correto da lógica e demonstrou, na prática, a complementaridade entre as duas linguagens: o Ladder para o comando booleano e o Structured Text para a supervisão com contagem e temporização.

Por fim, a discussão sobre controle preditivo e inteligência artificial situou o conhecimento adquirido no panorama das tecnologias emergentes, evidenciando que os fundamentos de modelagem, controle e análise dinâmica permanecem como base indispensável mesmo diante das abordagens mais avançadas.

REFERÊNCIAS

- BOLTON, William. Programmable Logic Controllers. 6. ed. Oxford: Newnes, 2015.
- FRANCHI, Claiton Moro; CAMARGO, Valter Luís Arlindo de. Controladores Lógicos Programáveis: Sistemas Discretos. 2. ed. São Paulo: Érica, 2009.
- INTERNATIONAL ELECTROTECHNICAL COMMISSION. IEC 61131-3: Programmable Controllers – Part 3: Programming Languages. Genebra: IEC, 2013.
- OGATA, Katsuhiko. Engenharia de Controle Moderno. 5. ed. São Paulo: Pearson, 2011.
- OPENPLC PROJECT. OpenPLC: Open Source PLC for Industrial and Home Automation. Disponível em: <https://openplcproject.com>. Acesso em: jun. 2026.

ROCKWELL AUTOMATION. The Connected Enterprise: Industrial Automation and Information. Milwaukee: Rockwell, 2020.